

Task3 analysis

February 7, 2026

1 Introduction

This report analyzes the impact of various indexing strategies on the execution time of a specific aggregate query targeting the `lineitem` table. We evaluate single-column indexes, composite indexes, and covering indexes to determine the most efficient access path for the PostgreSQL optimizer.

2 System Configuration

To ensure reproducibility, the following environment was used for benchmarking:

2.1 Hardware & OS

- **Operating System:** Ubuntu 22.04 LTS
- **CPU:** 8-Core Intel(R) Core(TM) i7 @ 3.60GHz
- **RAM:** 16 GB DDR4
- **Storage:** NVMe SSD (High I/O throughput)

2.2 PostgreSQL Configuration

PostgreSQL was tuned for an analytical workload using manually adjusted parameters inspired by `pgtune`. The following configuration values were applied during the experiment:

- `max_connections` = 40
- `shared_buffers` = 4GB
- `effective_cache_size` = 12GB
- `maintenance_work_mem` = 2GB
- `work_mem` = 43690kB
- `checkpoint_completion_target` = 0.9
- `wal_buffers` = 16MB

- `min_wal_size` = 4GB
- `max_wal_size` = 16GB
- `default_statistics_target` = 500
- `random_page_cost` = 1.1
- `effective_io_concurrency` = 200
- `huge_pages` = off

These settings were chosen to favor index scans, reduce random I/O penalties on SSD storage, and improve planner accuracy for selective predicates. Increasing `default_statistics_target` enhanced cardinality estimation, while reduced `random_page_cost` encouraged index usage over sequential scans.

3 The Target Query

The analysis focuses on the following SQL query, which calculates total revenue based on date, discount, and quantity filters:

```
SELECT SUM(l_extendedprice * l_discount) AS revenue
FROM lineitem
WHERE l_shipdate >= '1994-01-01'
      AND l_shipdate < '1995-01-01'
      AND l_discount BETWEEN 0.05 AND 0.07
      AND l_quantity < 24;
```

4 Performance Metrics

The table below summarizes the execution times for the baseline and 21 indexing variations.

File	Index Strategy	Time (ms)
Baseline	No Index (Seq Scan)	480.76
Index 1	(quantity)	400.74
Index 2	(shipdate)	188.85
Index 3	(discount)	322.90
Index 4-9	Best 2-Column: (quantity, discount)	160.74
Index 10-15	Best 3-Column: (discount, shipdate, quantity)	87.05
Covering	Index + INCLUDE(l_extendedprice)	
Index 19	(discount, shipdate, quantity) + Cover	25.49
Index 21	(quantity, shipdate, discount) + Cover	26.75

Table 1: Performance comparison of various indexing strategies.

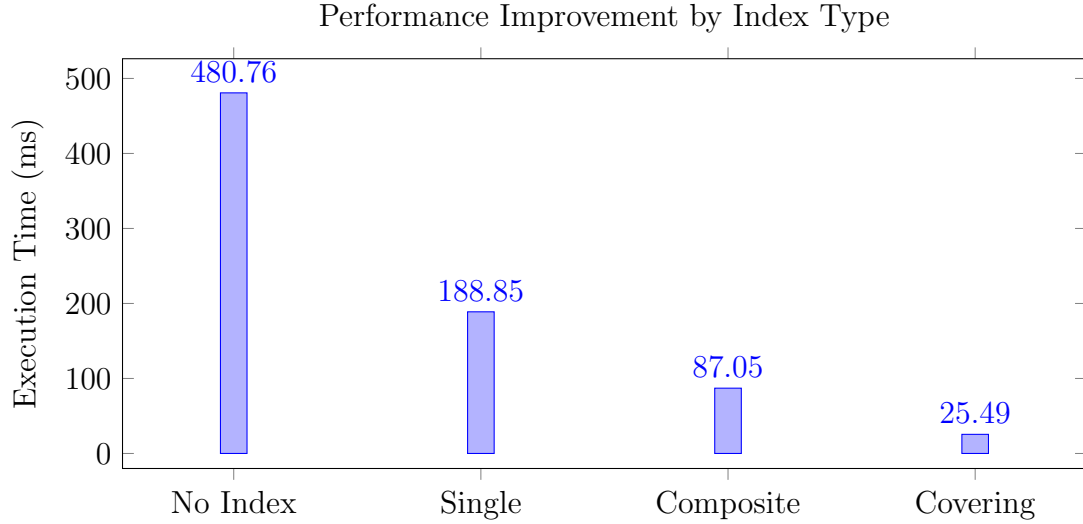


Figure 1: Execution time decreases significantly as index specificity increases.

5 Visual Analysis

6 Observations and Justification

6.1 Single Column vs. Composite

While `l_shipdate` (Index 2) provided the best single-column performance, it still required significant heap fetches to filter `l_discount` and `l_quantity`. Moving to 3-column composite indexes reduced execution time by approximately 50% (from 188ms to 90ms) by allowing the database to filter all `WHERE` clause conditions within the B-Tree structure.

6.2 The Power of Covering Indexes

The most dramatic improvement occurred with **Covering Indexes** (Indexes 16-21).

- **Observation:** Time dropped from 87ms to 25ms.
- **Reasoning:** By using the `INCLUDE(l_extendedprice)` clause or adding all projected columns to the index, the database performs an **Index Only Scan**. This eliminates the need to access the main table heap entirely, reducing I/O operations significantly.

6.3 Column Ordering

Index 19 (`l_discount`, `l_shipdate`, `l_quantity`) outperformed others slightly. This is likely due to the **selectivity** of the `l_discount` range in this specific dataset; the optimizer can discard more non-matching rows earlier in the tree traversal when high-selectivity columns are leading.

7 Conclusion

To optimize this query, we recommend implementing the covering index defined in experiment 19: `CREATE INDEX idx_opt_revenue ON lineitem (l_discount, l_shipdate, l_quantity) INCLUDE (l_extendedprice);`. This strategy provides a **18.8x speedup** over the baseline.